

**Part—7**

Lisp has been hailed as the world's most powerful programming language. But only the top percentile of programmers use it because of its cryptic syntax and academic reputation. This is rather unfortunate, since Lisp isn't that hard to grasp. If you want to be among the *crème de la crème*, this series is for you. This is the seventh article in the series, which began in the June 2011 issue of *LFY*.

Macros—can't get enough of them!

You already know from my previous articles that macros are Lisp programs that generate other Lisp programs. The generated Lisp code has fully parenthesised notation, and so does the macro that generates the code. In the simplest case, a macro substitutes forms within a template, clearly establishing a visual correspondence between the generating code and the generated code. Complex macros can use the full power of the Lisp language to generate code according to the macro parameters; often, a template form is wrapped in code that constructs appropriate sub-forms, but even this approach is just a typical pattern of use, and not a requirement (or restriction) of the Lisp macro facility.

To refresh our memories, let's examine the mechanism by which the Lisp system translates code generated by a macro. You define a macro with a *defmacro* form; *defmacro* is like *defun*, but instead of returning values, the body of the *defmacro* returns a Lisp form. Your program 'calls' a macro the same way it calls a function, but the behaviour is quite different. First, none of the macro's parameters are evaluated...ever. Macro parameters are bound literally to the corresponding arguments in the macro definition. If you pass `(* 7 (+ 3 2))` to a macro, the argument in the body of the macro definition is bound to the literal list `(* 7 (+ 3 2))` and not the value 35.

Next, the macro expander is invoked, receiving all of the actual parameters bound to their corresponding arguments as named by the *defmacro* form. The macro expander is merely the body of the *defmacro* form, which is just Lisp code; the only catch is that the Lisp system expects the macro expander to return a Lisp form. The Lisp system then evaluates whatever form the macro expander returns.

A Lisp implementation may expand macros at different times. A macro could be expanded just once, when your program is compiled. Or it could be expanded on first use as your program runs, and the expansion could be cached for subsequent reuse. A properly written macro will behave the same under all of these implementations.

Let's explore a real-world example of using a macro to extend Lisp into the problem domain. In addition to providing a macro expander, our new macro will automatically generate an environment that will be referenced by the expander. Our example will show how to move computations from run-time to compile-time, and how to share information computed at compile-time.

Let's say you're working on an interactive game that makes heavy use of the trigonometric function *sin r* in computing player motion and interaction. You've already determined that calling the Lisp function *sin* is too time-consuming; you also know that your program will work just fine with approximate results for the computation of